

Vite & Gourmand Documentation de déploiement

Démarche et étapes de mise en production

ECF Studi — TP Développeur Web et Web Mobile

Ce document décrit la démarche complète de déploiement de l'application Vite & Gourmand, de l'architecture cible jusqu'à la mise en ligne de chaque composant. L'application repose sur une architecture découplée : chaque brique (frontend, backend, bases de données) est hébergée sur un service spécialisé et déployée indépendamment.

1. Architecture de déploiement

L'application est distribuée sur quatre services cloud, chacun choisi pour son adéquation avec le composant hébergé et la disponibilité d'une offre gratuite adaptée à un projet d'examen :

Composant	Service	Rôle
Frontend SPA	Vercel	Hébergement statique + CDN + proxy /api/*
Backend API	Render (Docker)	Serveur applicatif Symfony
Base relationnelle	Clever Cloud	MySQL 8.0 managé
Base NoSQL	MongoDB Atlas	Statistiques de vente

L'ordre de déploiement recommandé est le suivant : d'abord les bases de données (elles doivent exister avant que le backend s'y connecte), puis le backend, et enfin le frontend.

2. Base de données relationnelle — Clever Cloud

Étapes (dashboard)

- Créer un compte sur Clever Cloud et accéder à la console.
- Créer un add-on MySQL (plan gratuit) et choisir la région.
- Une fois l'add-on créé, récupérer les informations de connexion dans l'onglet Informations : hôte, port, nom de la base, utilisateur, mot de passe.
- Construire la chaîne DATABASE URL au format Symfony :

```
mysql://USER:PASSWORD@HOST:PORT/BASE?serverVersion=8.0&charset=utf8mb4
```

Import du schéma (CLI)

- Appliquer les migrations Doctrine depuis le poste local, connecté à la base distante, ou laisser le backend les appliquer au premier déploiement :

```
php bin/console doctrine:migrations:migrate --no-interaction
```

- Charger les données de test :

```
mysql -u USER -p BASE < data_only.sql
```

⚠ Le plan gratuit Clever Cloud limite le nombre de connexions simultanées (5). Il convient d'en tenir compte pour la configuration du pool Doctrine.

3. Base NoSQL — MongoDB Atlas

Étapes (dashboard)

- Créer un compte MongoDB Atlas et un cluster gratuit M0.
- Dans Database Access, créer un utilisateur avec mot de passe.
- Dans Network Access, autoriser les adresses IP (0.0.0.0/0 pour un accès depuis Render, dont les IP sont dynamiques).
- Récupérer la chaîne de connexion (Connect > Drivers) :

```
mongodb+srv://USER:PASSWORD@CLUSTER.mongodb.net/
```

- Définir le nom de la base statistiques (ex. vite_gourmand_stats), utilisé par MONGO_DB.

⚠ La collection de statistiques est alimentée automatiquement par l'application (upsert à chaque commande) : aucune création manuelle de collection n'est nécessaire.

4. Backend API — Render (Docker)

Préparation du projet

- Le backend contient un Dockerfile à la racine, basant l'image sur php:8.4-apache et installant les extensions requises (mongodb, pdo_mysql, intl).
- Point clé : Apache doit écouter sur le port 10000, attendu par le scan de ports de Render (et non le port 80 par défaut). Ce réglage se fait dans la configuration Apache de l'image.
- Les clés JWT (private.pem / public.pem) ne sont pas commitées : elles sont fournies en base64 via des variables d'environnement, ou générées au démarrage.

Étapes (dashboard)

- Sur Render, créer un Web Service connecté au dépôt GitHub du backend.
- Choisir l'environnement Docker ; Render détecte le Dockerfile automatiquement.
- Sélectionner la branche main (déploiement automatique à chaque push).
- Renseigner toutes les variables d'environnement (section 6).
- Lancer le premier déploiement et suivre les logs de build.

⚠ Sur le plan gratuit, le service se met en veille après 15 minutes d'inactivité ; le premier appel suivant subit un délai de réveil de 30 à 60 secondes.

5. Frontend — Vercel (routing SPA + proxy API)

Préparation du projet

- Le frontend est une SPA statique (HTML/CSS/JS) : aucun build n'est nécessaire.
- Un fichier vercel.json à la racine remplit deux rôles, dans cet ordre : (1) proxy des appels API — toute requête /api/* reçue sur le domaine du front est réécrite côté serveur vers l'API Render, de sorte que les cookies JWT soient échangés en same-site (SameSite=Lax) malgré l'hébergement séparé ; (2) routing SPA — toutes les autres routes sont redirigées vers index.html pour que le routeur JavaScript maison prenne le relais.

```
{ "rewrites": [
  { "source": "/api/(.*)", "destination":
    "https://vite-gourmand-back-chap.onrender.com/api/$1" },
  { "source": "/images/(.*)", "destination":
    "https://vite-gourmand-back-chap.onrender.com/images/$1" },
  { "source": "/assets/(.*)", "destination":
    "https://vite-gourmand-back-chap.onrender.com/assets/$1" },
  { "source": "/bundles/(.*)", "destination":
    "https://vite-gourmand-back-chap.onrender.com/bundles/$1" },
  { "source": "/((?!.*\\.).*)", "destination": "/index.html" }
] }
```

⚠ L'ordre des règles est déterminant : le proxy /api/ — ainsi que /assets/*, les assets de Swagger UI étant servis par l'AssetMapper de Symfony (NelmioApiDocBundle v5) — doit précéder le catch-all SPA. Celui-ci exclut les chemins contenant un point pour ne pas intercepter les fichiers statiques (.css, .js, images). La règle /bundles/* est conservée par compatibilité avec l'ancienne convention de publication des assets. La règle /images/* relaie les images uploadées (menus) servies par le backend ; Vercel servant en priorité les fichiers réellement présents dans le déploiement, le dossier images/ statique du front n'est pas affecté — seuls les chemins absents du front sont relayés vers Render.*

Étapes (dashboard)

- Sur Vercel, importer le dépôt GitHub du frontend.

- Aucune configuration de build particulière : Vercel sert les fichiers directement.
- Sélectionner la branche main ; chaque push déclenche un redéploiement.
- Vérifier l'URL de production attribuée et l'utiliser comme origine autorisée côté backend (variable `CORS_ALLOW_ORIGIN`) pour les accès directs à l'API Render.

6. Variables d'environnement (Render)

Le backend est configuré exclusivement par variables d'environnement en production (aucun fichier .env.local n'est déployé). Les valeurs sensibles restent hors du dépôt Git.

Variable	Description
APP_ENV	prod
APP_SECRET	Clé secrète Symfony
DATABASE_URL	Connexion MySQL Clever Cloud
MONGO_URI / MONGO_DB	Connexion MongoDB + nom de base
MAILER_DSN	Transport Brevo
MAILER_SENDER_EMAIL / _NAME	Expéditeur des e-mails
CORS_ALLOW_ORIGIN	URL du frontend autorisée
JWT_SECRET_KEY / JWT_PUBLIC_KEY	Clés RSA (base64)
JWT_PASSPHRASE	Passphrase des clés JWT
ORS_API_KEY	Clé OpenRouteService

7. Vérification post-déploiement

- Accéder à l'URL du frontend et vérifier le chargement de la page d'accueil.
- Tester la connexion avec un compte de test (le premier appel peut être lent : réveil Render) et vérifier dans les outils du navigateur que les cookies sont bien posés en SameSite=Lax sur le domaine du front.
- Vérifier la documentation de l'API : /api/doc via le domaine du front (proxy) et via l'URL Render directe. Si Swagger UI reste réduit au seul lien NelmioApiDocBundle derrière le proxy, contrôler dans l'onglet Réseau que les fichiers /assets/* ne renvoient pas de 404 : c'est le signe que la règle /assets/* manque dans vercel.json.
- Effectuer un parcours complet : consultation menu, commande, changement de statut.
- Contrôler les en-têtes de sécurité et la politique CORS depuis les outils du navigateur.

⚠ Limitation connue : le stockage de fichiers sur Render est éphémère ; les images uploadées sont perdues au redéploiement. Acceptable dans le cadre de l'ECF ; une solution de stockage externe (S3 ou équivalent) serait l'évolution recommandée en production réelle.